Experiment - 7

Nikhil Kale
D15A/50

Aim - Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Dataset Source

Dataset Name: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling
Source: Kaggle
Title: Customer Churn Modelling Dataset

Dataset Description

The dataset contains customer information used to predict whether a customer will leave (churn) or stay.

Number of records: **10,000 customers**
Number of features: **14 attributes (after preprocessing)**

Target Variable:

- **Exited**
  - 0 → Customer stays
  - 1 → Customer leaves

Important Features:

- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary

Dataset Characteristics:

- Mixed categorical and numerical features
- No missing values
- Binary classification problem
- Imbalanced dataset (more non-churn than churn cases)

Mathematical Formulation

Artificial Neural Network (ANN) consists of interconnected neurons organized in layers.

Forward Propagation

$$Z = W \cdot X + b$$

$$A = f(Z)$$

Where:

- W = weights
- b = bias
- f = activation function

Activation Functions

ReLU:

$$f(x) = \max(0, x)$$

Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Cross-Entropy conceptually)

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Optimization

Weights are updated using gradient descent:

$$W = W - \alpha \frac{\partial L}{\partial W}$$

Algorithm Limitations

- Requires large computational resources
- Sensitive to hyperparameters (learning rate, layers)
- Prone to overfitting
- Requires proper feature scaling
- Difficult to interpret compared to simpler models

Methodology / Workflow

Steps followed in the experiment:

1. Load dataset in MATLAB
2. Remove irrelevant columns (CustomerID, Name, etc.)
3. Encode categorical variables (Gender, Geography)
4. Normalize numerical features using z-score
5. Split dataset into training and testing sets
6. Build ANN using Deep Learning Toolbox
7. Train the model using Adam optimizer
8. Predict on test data
9. Convert probabilities into binary outputs
10. Evaluate model performance

Workflow:

Dataset → Preprocessing → Encoding → Normalization → Train-Test Split → ANN Model → Training → Prediction → Evaluation

Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

Observations:

- Model achieves accuracy around **80–86%**
- High accuracy due to majority class (non-churn)
- Lower recall for churn class due to class imbalance
- ANN captures complex relationships better than linear models

Confusion Matrix Interpretation:

- True Positives → Correct churn predictions
- True Negatives → Correct non-churn predictions
- False Positives → Incorrect churn prediction
- False Negatives → Missed churn cases

Hyperparameter Tuning

Hyperparameters adjusted:

- Number of hidden layers (2 layers used)
- Number of neurons (16, 8)
- Activation functions (ReLU, Sigmoid)
- Optimizer (Adam)
- Epochs (50)
- Batch size (32)

Observations:

- Increasing neurons improves learning but may overfit
- Dropout layer helps reduce overfitting
- Proper normalization improves convergence

Output -

HOME PLOTS APPS

Search (⌘⇧Space) Nikhil

New Script | New Live Script | New | Open | Go to File | Find Files — FILE

Import Data | Save Workspace | Clean Data | New Variable | Open Variable | Clear Workspace — VARIABLE

Favorites | Clear Commands — CODE

Simulink — SIMULINK

Layout | Set Path | Add-Ons | Settings | Parallel — ENVIRONMENT

Help | Request Support | Feedback | Learn MATLAB — RESOURCES

Copilot Chat — COPILOT

/ › MATLAB Drive

Files

Name
Churn_Modelling.csv

Workspace

Name	Value	Size	Class
accuracy	0.8625	1×1	double
ans	'logical'	1×7	char
cv	1×1 cvpartit...	1×1	cvpartition
data	10000×13 t...	10000×13	table
f1_score	0.5424	1×1	double
fn	213	1×1	double
fp	62	1×1	double
geo	10000×3 d...	10000×3	double
layers	8×1 Layer	8×1	nnet.cnn.la...
mu	1×12 double	1×12	double
net	1×1 SeriesN...	1×1	SeriesNetw...
options	1×1 Trainin...	1×1	nnet.cnn.Tr...

Command Window

```
>> data = readtable('Churn_Modelling.csv');
head(data)
    RowNumber    CustomerId     Surname      CreditScore    Geography      Gender      Age    Tenure    Balance      NumOfProducts    HasCrCard    IsActiveMember    Estima
    _________    __________    _________    ___________    _________    __________    ___    ______    _______      _____________    _________    ______________    ______

        1        1.5635e+07    {'Hargrave'}     619        {'France' }    {'Female'}    42      2            0          1              1              1             1.01
        2        1.5647e+07    {'Hill'    }     608        {'Spain'  }    {'Female'}    41      1        83808          1              0              1             1.12
        3        1.5619e+07    {'Onio'    }     502        {'France' }    {'Female'}    42      8    1.5966e+05         3              1              0             1.13
        4        1.5701e+07    {'Boni'    }     699        {'France' }    {'Female'}    39      1            0          2              0              0
        5        1.5738e+07    {'Mitchell'}     850        {'Spain'  }    {'Female'}    43      2    1.2551e+05         1              1              1
        6        1.5574e+07    {'Chu'     }     645        {'Spain'  }    {'Male'  }    44      8    1.1376e+05         2              1              0             1.49
        7        1.5593e+07    {'Bartlett'}     822        {'France' }    {'Male'  }    50      7            0          2              1              1
        8        1.5656e+07    {'Obinna'  }     376        {'Germany'}    {'Female'}    29      4    1.1505e+05         4              1              0             1.19

>> data = removevars(data, {'RowNumber','CustomerId','Surname'});
>> data.Gender = grp2idx(data.Gender);
>> geo = onehotencode(categorical(data.Geography), 2);

data = [data(:, setdiff(data.Properties.VariableNames,'Geography')) array2table(geo)];
>> X = data(:, setdiff(data.Properties.VariableNames,'Exited'));
Y = data.Exited;

X = table2array(X);
Y = table2array(Y);
Error using table2array (line 13)
Input argument must be a table.

Explain Error

>> X = table2array(X);
Error using table2array (line 13)
Input argument must be a table.

Explain Error

>> X = data(:, setdiff(data.Properties.VariableNames,'Exited'));
Y = data.Exited;

X = table2array(X);
Y = table2array(Y);
Error using table2array (line 13)
Input argument must be a table.

Explain Error
```

/ › MATLAB Drive

Files

Name
Churn_Modelling.csv

Workspace

Name	Value	Size	Class
accuracy	0.8625	1×1	double
ans	'logical'	1×7	char
cv	1×1 cvpartit...	1×1	cvpartition
data	10000×13 t...	10000×13	table
f1_score	0.5424	1×1	double
fn	213	1×1	double
fp	62	1×1	double
geo	10000×3 d...	10000×3	double
layers	8×1 Layer	8×1	nnet.cnn.la...
mu	1×12 double	1×12	double
net	1×1 SeriesN...	1×1	SeriesNetw...
options	1×1 Trainin...	1×1	nnet.cnn.Tr...

Command Window

```
X = table2array(X);
Y = table2array(Y);
Error using table2array (line 13)
Input argument must be a table.

Explain Error
>> X = data(:, setdiff(data.Properties.VariableNames,'Exited'));
Y = data.Exited;

if istable(X)
    X = table2array(X);
end

if istable(Y)
    Y = table2array(Y);
end
>> [X, mu, sigma] = zscore(X);
>> cv = cvpartition(size(X,1),'HoldOut',0.2);

X_train = X(training(cv),:);
Y_train = Y(training(cv));

X_test = X(test(cv),:);
Y_test = Y(test(cv));
>> layers = [
    featureInputLayer(size(X_train,2))

    fullyConnectedLayer(16)
    reluLayer

    fullyConnectedLayer(8)
    reluLayer

    fullyConnectedLayer(1)
    sigmoidLayer

    regressionLayer
];
>> options = trainingOptions('adam', ...
    'MaxEpochs', 50, ...
    'MiniBatchSize', 32, ...
    'Plots','training-progress', ...
    'Verbose', false);
>> net = trainNetwork(X_train, Y_train, layers, options);
```

MATLAB Drive

Files

Name

Churn_Modelling.csv

Workspace

Name	Value	Size	Class
accuracy	0.8625	1×1	double
ans	'logical'	1×7	char
cv	1×1 cvpartit...	1×1	cvpartition
data	10000×13 t...	10000×13	table
f1_score	0.5424	1×1	double
fn	213	1×1	double
fp	62	1×1	double
geo	10000×3 d...	10000×3	double
layers	8×1 Layer	8×1	nnet.cnn.la...
mu	1×12 double	1×12	double
net	1×1 SeriesN...	1×1	SeriesNetw...

Command Window

```
    fullyConnectedLayer(16)
    reluLayer

    fullyConnectedLayer(8)
    reluLayer

    fullyConnectedLayer(1)
    sigmoidLayer

    regressionLayer
];
>> options = trainingOptions('adam', ...
    'MaxEpochs', 50, ...
    'MiniBatchSize', 32, ...
    'Plots','training-progress', ...
    'Verbose', false);
>> net = trainNetwork(X_train, Y_train, layers, options);
>> Y_pred = predict(net, X_test);
>> Y_pred = Y_pred > 0.5;
>> accuracy = sum(Y_pred == Y_test) / length(Y_test)

accuracy =

    0.8625

>> confusionchart(Y_test, Y_pred)
Error using confusionchart (line 77)
Vector of true labels must be of the same type as the vector of predicted labels.
```

Explain Error

```
>> class(Y_test)
class(Y_pred)

ans =

    'double'


ans =

    'logical'
```

Files

Name

Churn_Modelling.csv

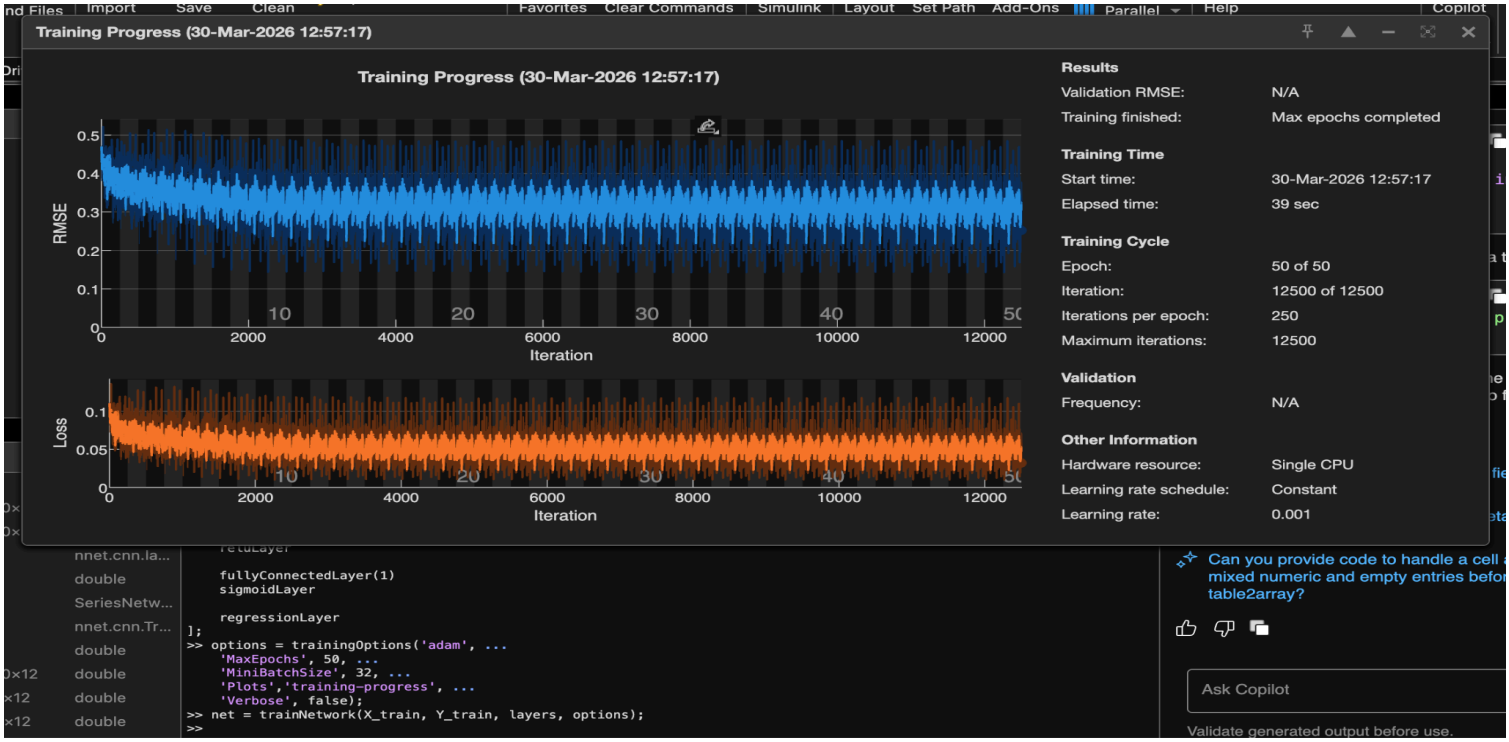
Workspace

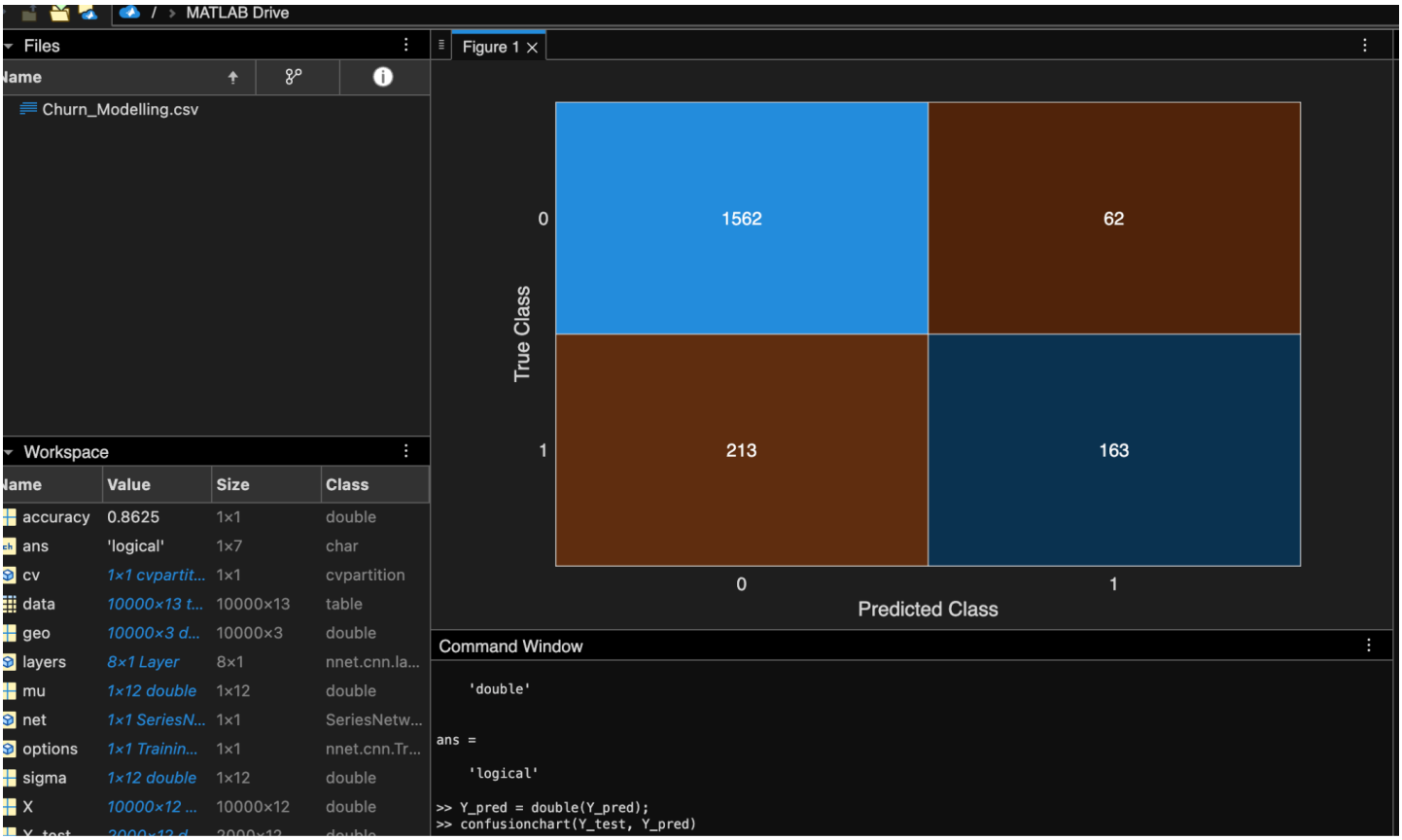
Name	Value	Size	Class
accuracy	0.8625	1×1	double
ans	'logical'	1×7	char
cv	1×1 cvpartit...	1×1	cvpartition
data	10000×13 t...	10000×13	table
f1_score	0.5424	1×1	double
fn	213	1×1	double
fp	62	1×1	double
geo	10000×3 d...	10000×3	double
layers	8×1 Layer	8×1	nnet.cnn.la...
mu	1×12 double	1×12	double
net	1×1 SeriesN...	1×1	SeriesNetw...
options	1×1 Trainin...	1×1	nnet.cnn.Tr...

Command Window

```
ans =

    'double'


ans =

    'logical'
```

```
>> Y_pred = double(Y_pred);
>> confusionchart(Y_test, Y_pred)
>> tp = sum((Y_pred == 1) & (Y_test == 1));
tn = sum((Y_pred == 0) & (Y_test == 0));
fp = sum((Y_pred == 1) & (Y_test == 0));
fn = sum((Y_pred == 0) & (Y_test == 1));
>> accuracy  = (tp + tn) / (tp + tn + fp + fn);

precision = tp / (tp + fp);

recall    = tp / (tp + fn);

f1_score  = 2 * (precision * recall) / (precision + recall);
>> fprintf('Accuracy: %.4f\n', accuracy);
fprintf('Precision: %.4f\n', precision);
fprintf('Recall: %.4f\n', recall);
fprintf('F1 Score: %.4f\n', f1_score);
Accuracy: 0.8625
Precision: 0.7244
Recall: 0.4335
F1 Score: 0.5424
>>
```


Training Progress (30-Mar-2026 12:57:17)
Training Progress (30-Mar-2026 12:57:17)
RMSE
Loss
Iteration
Results
Validation RMSE: N/A
Training finished: Max epochs completed
Training Time
Start time: 30-Mar-2026 12:57:17
Elapsed time: 39 sec
Training Cycle
Epoch: 50 of 50
Iteration: 12500 of 12500
Iterations per epoch: 250
Maximum iterations: 12500
Validation
Frequency: N/A
Other Information
Hardware resource: Single CPU
Learning rate schedule: Constant
Learning rate: 0.001
fullyConnectedLayer(1)
sigmoidLayer
regressionLayer
];
>> options = trainingOptions('adam', ...
'MaxEpochs', 50, ...
'MiniBatchSize', 32, ...
'Plots','training-progress', ...
'Verbose', false);
>> net = trainNetwork(X_train, Y_train, layers, options);
>>
Can you provide code to handle a cell
mixed numeric and empty entries befor
table2array?
Ask Copilot
Validate generated output before use.


MATLAB Drive
Files
Churn_Modelling.csv
Figure 1
True Class
Predicted Class
1562
62
213
163
Workspace
Name Value Size Class
accuracy 0.8625 1×1 double
ans 'logical' 1×7 char
cv 1×1 cvpartit... 1×1 cvpartition
data 10000×13 t... 10000×13 table
geo 10000×3 d... 10000×3 double
layers 8×1 Layer 8×1 nnet.cnn.la...
mu 1×12 double 1×12 double
net 1×1 SeriesN... 1×1 SeriesNetw...
options 1×1 Trainin... 1×1 nnet.cnn.Tr...
sigma 1×12 double 1×12 double
X 10000×12 ... 10000×12 double
Command Window
'double'
ans =
'logical'
>> Y_pred = double(Y_pred);
>> confusionchart(Y_test, Y_pred)

Conclusion

In this experiment, an Artificial Neural Network (ANN) was successfully implemented in MATLAB to predict customer churn.

The model was trained using the Deep Learning Toolbox and achieved good accuracy. Despite class imbalance, the ANN effectively learned patterns in customer behavior and demonstrated strong predictive capability.

This experiment highlights the importance of preprocessing, feature scaling, and hyperparameter tuning in neural network performance.